GABRIEL POPA

Senior Software Engineer – Rust (Blockchain/Web3) • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | https://www.linkedin.com/in/gabriel-popa-12890b414/

ABOUT ME

Senior Software Engineer with 10+ years of experience building backend platforms, distributed services, and high-reliability product systems, now specialized in **Rust-based Web3 and blockchain engineering** across infrastructure, indexers, validator-facing tooling, and transaction-heavy data services. Strongest work combines **Rust 1.56–1.75+**, **Tokio 1.x**, **Axum / Actix Web**, **PostgreSQL**, **Redis**, **Docker**, and **AWS** to deliver secure, low-latency systems for blockchain analytics, wallet flows, staking operations, and on-chain data processing in the **fintech and decentralized technology industry**.

SKILLS

- **Languages: Rust 1.56–1.75+**, TypeScript, JavaScript, Python, SQL, Bash
- **Blockchain / Web3: Solana**, **Ethereum**, **EVM**, JSON-RPC, **Substrate** basics, smart contract integration, transaction indexing, wallet flows, staking systems, block ingestion, event decoding, explorer-style APIs, signing flows, token transfer monitoring, on-chain analytics, mempool-aware processing concepts
- **Backend & APIs: Tokio 1.x**, **Axum 0.6–0.7**, **Actix Web 4.x**, async services, REST APIs, WebSocket streams, background workers, event-driven processing, service decomposition, auth flows, rate limiting, idempotent job handling
- **Data & Messaging: PostgreSQL**, MySQL, Redis, Kafka, RabbitMQ, Elasticsearch, ClickHouse basics, SQL optimization, caching strategies, indexing, reconciliation jobs, stream consumers
- **Cloud & DevOps: AWS**, Docker, Kubernetes, Helm, GitHub Actions, GitLab CI, Terraform basics, Linux, Nginx, CI/CD pipelines, containerized deployments, release automation
- **Observability / Quality / Security:** OpenTelemetry basics, Prometheus, Grafana, structured logging, tracing, unit testing, integration testing, load-aware debugging, secure secret handling, JWT, RBAC, audit logging, incident support

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build, Remote *(Jan2024 – Dec 2025)*

- Delivered backend services for a **blockchain operations platform** using **Rust**, **Tokio**, **Axum**, **PostgreSQL**, and **Redis**, supporting staking dashboards, transaction monitoring, wallet-linked administration, and internal reconciliation workflows with stronger throughput under production load.
- Led migration of latency-sensitive background workers from older **Node.js** services into **Rust 1.74+** async jobs, which reduced runtime overhead, improved concurrency handling, and cut peak processing delays for chain event ingestion by **32%**.
- Built a resilient on-chain indexing service that consumed **JSON-RPC** data, normalized block and transaction payloads, and persisted chain activity into **PostgreSQL**, giving product teams a stable source for explorer-style APIs and operational reporting.
- Solved an intermittent duplication bug where retry logic replayed the same transfer event during provider instability, then fixed it with idempotency keys, safer confirmation thresholds, and checkpoint-aware consumers that reduced reconciliation noise by **41%**.
- Implemented a new wallet activity module that tracked balances, staking actions, fee movements, and delegated validator events, allowing support teams to investigate account behavior without querying raw chain data manually.
- Refactored oversized database access code into smaller repository and service layers, making transaction-scoped business rules easier to test and reducing regression risk when adding new chain-specific parsing behavior.
- Improved heavy API response times by profiling slow joins, adding targeted indexes, and splitting synchronous processing from non-critical enrichments, which lowered median response time on high-traffic endpoints by **29%**.
- Introduced structured logging and tracing around block ingestion, confirmation processing, and payout reconciliation, making it easier to isolate provider-side failures and shorten production investigation cycles during network volatility.
- Stabilized a memory growth issue in a long-running stream processor by removing unnecessary payload cloning, tightening buffer lifetimes, and batching writes more carefully, which improved runtime stability during high block volume windows.

- Worked closely with product and DevOps teams to release secure operational features such as role-restricted wallet controls, audit-friendly action records, and safer admin approval flows for sensitive blockchain operations.
- Containerized Rust services with **Docker** and deployed them through CI pipelines into cloud-hosted environments, improving release consistency and making rollback handling more predictable when provider integrations behaved unexpectedly.
- Mentored other engineers on practical **Rust** patterns such as ownership-friendly service boundaries, async error propagation, and trait-based abstractions, helping the team adopt the language without overengineering simpler business modules.

Software Engineer

Next Apps, Remote *(Nov2020 – Nov 2023)*

- Built backend services and integration workflows for **Web3-enabled financial products**, combining **Rust**, **TypeScript**, **PostgreSQL**, **Redis**, and API-layer orchestration to support wallet connectivity, transaction history, and token movement visibility.
- Contributed to early **Rust 1.60–1.70** adoption by moving compute-heavy parsing and validation routines out of slower application paths, which improved consistency for chain event processing and reduced CPU spikes during batch imports by **24%**.
- Developed APIs that translated noisy **Ethereum** and **EVM-compatible** provider responses into cleaner business-ready payloads, allowing frontend teams to ship more reliable user-facing views for balances, transfers, and contract-driven activity.
- Resolved a recurring production issue where delayed confirmations caused misleading status transitions in user dashboards, then corrected it by separating submitted, seen, confirmed, and finalized states with clearer retry and timeout behavior.
- Implemented WebSocket-backed notification flows for wallet activity and settlement progress, reducing manual refresh behavior and giving operations users faster visibility into chain-dependent events that previously appeared inconsistent.
- Improved internal monitoring by adding richer logs, alert-friendly metrics, and chain/provider error categorization, which made it easier to distinguish external RPC instability from application defects during incident response.
- Supported migration of an older service layer into more modular backend components, replacing tightly coupled request handlers with clearer job workers and queue-backed tasks that handled backfills and delayed reconciliation more safely.
- Built reconciliation jobs that compared internal ledger records with chain-derived outcomes, helping finance and support teams detect mismatches earlier and reducing time spent on manual investigation of failed or partial transfers.
- Enhanced SQL performance for account and transaction views by reducing overfetching, tuning pagination behavior, and revisiting indexes for high-volume query paths, which improved dashboard responsiveness by **27%** on busy customer accounts.
- Partnered with frontend engineers on API contract stability, error shape consistency, and pagination rules so new wallet and staking screens could move faster without repeated backend clarification during active sprint delivery.
- Helped newer team members learn blockchain-specific backend concerns such as finality windows, duplicate event protection, RPC variability, and safe retry design, improving shared engineering confidence around production Web3 behavior.

Software Developer

Datamart, Remote *(Feb 2016 – Sept 2020)*

- Developed backend-heavy data products using **Python**, **Node.js**, **SQL**, **PostgreSQL**, and reporting pipelines, building the event processing, data modeling, and operational debugging habits that later transferred well into blockchain data engineering.
- Built ingestion jobs for large transactional datasets, normalized inconsistent upstream payloads, and improved persistence logic for analytics workflows, which reduced downstream reporting corrections and improved trust in platform-generated numbers.
- Solved a recurring import error caused by malformed external records and timezone drift, then introduced safer validation, clearer exception handling, and quarantine logic so bad payloads no longer blocked healthy batches.
- Created internal APIs and scheduled workers for reconciliation-style business processes, giving operations teams better visibility into late records, partial failures, and data mismatches that previously required manual spreadsheet checks.
- Improved slower SQL paths by revisiting join order, reducing unnecessary scans, and adding practical indexes for reporting tables, which shortened runtime for some nightly data jobs by **31%** as volumes continued to grow.

- Helped modernize deployment practices by containerizing selected internal services and documenting safer release routines, which reduced environment inconsistencies and made issue reproduction much easier across development and staging systems.
- Contributed to early high-performance utility work by using **Rust** for small data transformation tools near the end of the role, mainly where memory safety and predictable speed were valuable for parsing and reshaping large file-based inputs.
- Investigated several production support issues where background jobs silently failed after partial success, then improved logging depth, failure visibility, and restart handling so support teams could react before customers noticed missing outputs.
- Worked with product and reporting stakeholders to turn vague data requests into more stable service and schema changes, balancing delivery speed with safer rollout practices on systems that were still evolving quickly.
- Supported junior-level mentoring informally by sharing debugging notes, SQL tuning examples, and release checklists that helped newer teammates avoid common mistakes in data-heavy backend workflows.

Junior Software Developer

Berg Software, Remote *(Sep2014 – Mar 2016)*

- Supported web application maintenance using **JavaScript**, **Python**, **SQL**, **MySQL**, and server-side business logic, contributing bug fixes, validation improvements, and smaller backend enhancements under guidance from senior engineers.
- Fixed a recurring form submission defect caused by mismatched client and server validation rules, then aligned error handling across both layers so fewer invalid requests reached persistence paths and user frustration dropped noticeably.
- Built small internal modules for data entry, lookup screens, and administrative updates, learning how to translate business rules into maintainable code while keeping risk low on frequently used operational pages.
- Improved a slow record search page by simplifying query construction, reducing unnecessary payload fields, and tightening pagination behavior, which made navigation feel more stable for users handling larger account datasets.
- Investigated an intermittent login/session issue by tracing cookie handling, timeout behavior, and request flow, then supported a safer fix that made authentication behavior more predictable across browser sessions.
- Assisted with release preparation, basic deployment checks, and issue reproduction steps, helping the team catch several environment-specific problems before production rollout and improving confidence in small but frequent releases.
- Wrote reusable helper functions and cleaned duplicated logic in older modules, which made later bug fixes easier to apply and gave the team a more maintainable base for incremental feature work.
- Helped correct problematic records with targeted SQL scripts and safer update procedures, preventing repeated application errors caused by legacy data inconsistencies that could otherwise break common user workflows.
- Gained strong grounding in production debugging, incremental delivery, and backend discipline by working through real support issues, code reviews, and maintenance tasks that required careful changes rather than risky rewrites.

EDUCATION

Bachelor’s Degree in Computer Science

University of Bucharest | (*2010 – 2014)*

- Academic foundation covered software engineering, algorithms, database systems, and practical application development, providing the base for long-term specialization in backend engineering and scalable service design.

CERITIFICATIONS

- **Rust for Blockchain Development — 2024**
  Demonstrated practical capability in building high-performance blockchain services with **Rust**, including async processing, memory-safe backend design, and reliable transaction-oriented system behavior.
- **Docker & Kubernetes Foundations — 2023**
  Validated working knowledge of containerized deployment, runtime isolation, and basic orchestration patterns commonly used for modern backend service delivery.
- **PostgreSQL for Developers — 2022**
  Demonstrated hands-on understanding of relational modeling, query optimization, indexing, and performance-aware backend data access patterns.